Table des matières

Exercices corrigés 05 — Tester, historiser et documenter 1
Questions courtes 1
Questions pratiques 3
Diagnostics 3
Questions à développer 4
Mises en situation 4
Corrigé 5

Exercices corrigés 05 — Tester, historiser et documenter

Comment travailler ces exercices. Répondez seul, puis vérifiez avec le corrigé. Les exercices marqués [labo] portent sur ce que vous venez de faire au labo ; les autres vérifient que le cours est clair dans votre tête. Tout vient de la Théorie 05 et du Labo 05.

Le corrigé suit les énoncés, en fin de document. Répondez à l'énoncé en entier avant d'y passer : c'est en butant sur une question qu'on découvre ce qu'on avait lu sans le comprendre.

Questions courtes

1. La définition. Dites ce qu'est un test dbt, en une phrase, et nommez deux choses qu'il n'est pas.
2. Les trois sortes. Nommez les trois sortes de tests de dbt, et dites où chacune se déclare.
3. Le nom du bloc. Donnez le nom du bloc qui porte les tests dans un fichier YAML, et dites ce que l'ancien nom produit aujourd'hui.
4. Les quatre génériques. Nommez les quatre tests génériques livrés avec dbt, et dites sur quoi le cours pose chacun chez Boréal.
5. [labo] Le test qui ment. Dites lequel des quatre passe au vert sur un décodage entièrement raté, pourquoi, le compte que vous avez vu sortir, et la parade.
6. Le trou rebouché. Dites ce que `relationships` garantit, et quel choix du Cours 04 le rend obligatoire plutôt que facultatif.
7. Le grain, une fois pour toutes. Nommez le test de `dbt_utils` qui vérifie le grain d'un fait, et dites ce qu'il remplace.

© 2026 iAlexMG. Tous droits réservés.

8. Le test singulier. Dites où vit un test singulier, ce que dbt attend de sa requête, et pourquoi la réconciliation de Boréal ne peut pas être un test générique.
9. [labo] La jointure aveugle. Dites ce qu'une jointure interne ne voit pas dans une réconciliation, donnez les deux comptes que vous avez mesurés sur le fait amputé, et nommez les trois cas qu'une réconciliation doit sortir.
10. Les trois réglages. Nommez les trois réglages qui changent le comportement d'un test qui échoue, et dites ce que chacun fait.
11. [labo] Le seuil. Donnez la valeur du seuil posé sur `annee_modele`, la raison de cette valeur, et dites ce qu'un seuil documente qu'un test retiré ne documente pas.
12. Le test unitaire. Nommez ses trois parties, dites ce qu'il attrape qu'aucun test de données n'attrape, et ce qui arrive aux colonnes que `expect` ne cite pas.
13. Le modèle invisible. Dites pourquoi un test unitaire échoue quand une de ses entrées est un modèle `ephemeral`, et donnez la parade.
14. Le snapshot. Dites ce que dbt fait à chaque `dbt snapshot`, nommez les quatre colonnes qu'il pose, et dites ce que chacune porte.
15. Les deux stratégies. Nommez les deux stratégies de détection d'un changement, dites celle que Boréal emploie, et pourquoi.
16. [labo] La date des versions. Dites ce que dbt fait sans la configuration `updated_at`, ce qu'il fait avec, et pourquoi le cas de Boréal impose la seconde.
17. La table et la vue. Dites en quoi un snapshot diffère d'un modèle ordinaire quant au moment où il change, et donnez la conséquence que le labo vous a fait constater sur un staging cassé.
18. La fréquence. Chez Boréal, la révision des paliers a eu lieu le 1er juillet 2024 et le snapshot la date du 31. Dites pourquoi, ce que cela change pour les ventes du mois, et le montant.
19. Les trois décisions du type 2. Dites sur quoi se hache la clé d'une dimension de type 2, ce qu'on fait des colonnes qu'on ne suit pas, et à quelle date s'ouvre la première version de chaque entité.

1. La jointure temporelle. Donnez la forme de l'intervalle, et dites ce que chacun des deux contrôles du Cours 04 attrape quand elle est fausse.
2. Ce qui change et ce qui ne change pas. Dites ce que l'historisation change dans l'étoile, ce qu'elle ne change pas, et donnez le chiffre que le labo a mesuré chez Boréal.
3. La documentation. Nommez les trois fichiers que `dbt docs generate` produit, dites ce que chacun porte, et nommez les deux usages du `manifest.json` dans la suite du parcours.
4. L'`exposure` et le `rpt_`. Dites ce qu'un `exposure` déclare et qui doit en être le propriétaire, puis nommez les trois choses qu'un modèle `rpt_*` a le droit de faire.

© 2026 iAlexMG. Tous droits réservés.

Questions pratiques

P1. [labo] Tester une dimension qui a changé de forme. La dimension du concessionnaire porte maintenant une ligne par version. Dites ce qu'un `unique` posé sur `code_concessionnaire` rendrait, donnez le compte, et écrivez le test qui vérifie réellement le grain de cette dimension.

P2. [labo] La réconciliation. Écrivez le test singulier qui compare la valeur nette du fait à celle des en-têtes de commande. Dites ce que chacune de ses trois particularités règle, et donnez le résultat attendu.

P3. Le coût du type 1, mesuré après coup. Maintenant que la dimension porte l'histoire, écrivez la requête qui compte les lignes de vente dont le palier d'époque diffère du palier d'aujourd'hui, avec leur valeur, année par année. Donnez le résultat et dites ce que la dernière année vous apprend.

P4. Un test de plus. La dimension ne doit ni laisser de trou ni faire chevaucher deux versions d'un même concessionnaire. Écrivez le test singulier qui le vérifie, donnez l'attendu, et dites ce que les tests déjà en place feraient de ces deux défauts.

P5. (avancé) Le test unitaire du plancher. Écrivez le test unitaire qui vérifie que la première version d'un concessionnaire s'ouvre au plancher et la seconde à sa vraie date. Dites combien d'entrées il faut donner et pourquoi, et ce que dbt répond quand on en oublie une.

P6. Ce que je casse, et d'où ça vient. Donnez la commande qui liste tout ce qui dépend de `dim_concessionnaire`, et celle qui liste tout ce dont le tableau de bord des ventes dépend. Donnez les deux comptes, et dites à quelle question chacune répond.

P7. Le taux de marge par palier. Depuis le résumé de la couche `PRESENTATION`, écrivez la requête qui donne le taux de marge par palier pour 2025. Donnez le résultat, puis celui qu'on obtient en moyennant les taux mensuels, et dites lequel est juste.

P8. (avancé) Rejouer les quarante-neuf lots. Le labo n'en rejoue que quatre. Dites comment rejouer les 49, ce que cela change dans le snapshot, ce que cela change dans l'étoile finale, et ce que cela coûte.

Diagnostics

E1. L'histoire datée d'aujourd'hui. Un collègue reprend le snapshot et retire la ligne `updated_at='date_extrait'`, en expliquant que dbt sait bien dater ses versions tout seul. Il rejoue les quatre lots, reconstruit l'étoile, et tout passe au vert : 35 tests, réconciliation à 0,00 $ d'écart. Dites ce que porte le snapshot, ce que le tableau des ventes 2025 par palier annonce désormais, et pourquoi aucun test ne l'a vu.

E2. Le lot rejoué trop tard. Un collègue construit le projet par un `dbt build` complet, sans exclure le snapshot, avant d'avoir rejoué le moindre lot. Puis il suit le labo et lance `dbt snapshot --vars "{lot: 2024-06}"`. Dites ce que le snapshot contient ensuite, donnez les comptes, et dites ce qui, dans la mécanique de la stratégie `check`, produit ce résultat.

E3. Les lignes qui manquent, puis celles qui restent. Un collègue écrit la jointure temporelle avec l'intervalle semi-ouvert, mais sans le plancher de la première version. Il lance `dbt build --exclude snp_concessionnaire` : un seul test tombe. Il remplace alors la jointure interne par un `left join`, « pour ne plus

© 2026 iAlexMG. Tous droits réservés.

perdre de lignes », et relance. Dites ce qui tombe dans chacun des deux cas, avec les comptes, et dites ce que l'histoire enseigne sur la complémentarité des tests.

E4. Le rouge qu'on met en jaune. Un collègue, pressé de livrer, met la réconciliation en `severity: warn` « le temps de trouver d'où vient l'écart ». Le pipeline de nuit tourne depuis trois semaines. Dites ce que dbt affiche, ce que l'ordonnanceur en conclut, et ce qu'il aurait fallu faire à la place.

E5. La dimension qui triple. Un collègue trouve le snapshot trop avare et ajoute `date_extrait` aux colonnes suivies, « pour garder la trace de chaque extrait ». Il rejoue les quatre lots et reconstruit. Tous les tests passent, le chiffre d'affaires par palier est inchangé au cent près. Donnez le nombre de versions, dites ce qui s'est passé, et dites où le défaut se voit puisque aucun test ne le signale.

Questions à développer

A. Combien de tests est-ce assez. On vous reproche vingt-quatre tests sur douze modèles, « du temps passé à tester des évidences ». Défendez le nombre, dites ce qui distingue un test qui gagne sa place d'un test qui remplit un tableau de bord de qualité, et nommez ce que vos tests actuels ne vérifient toujours pas.

B. Type 1 ou type 2. On vous demande d'historiser toutes les colonnes de toutes les dimensions, « parce qu'on ne sait jamais ». Défendez la position inverse, dites ce qui décide qu'une colonne mérite d'être suivie, et ce que vous écrivez pour que le choix reste défendable dans deux ans.

C. (avancé) Le jour où les clés changent. Le passage en type 2 a refait toutes les clés de `dim_concessionnaire`. Dites ce que cela casse chez ceux qui consomment déjà l'étoile, ce qui ne casse pas, et la façon de mener ce changement pour qu'il ne se découvre pas un lundi matin dans un rapport vide.

Mises en situation

M1. Distribution alimentaire de Saint-Raymond. L'entreprise a une étoile des ventes bâtie l'an dernier par un consultant, qui tourne tous les soirs et qu'aucun test ne couvre. On vous donne une journée pour la rendre défendable. Dites les tests que vous posez en premier et dans quel ordre, celui que vous ne poserez pas tout de suite, et ce que vous rendez à la fin de la journée.

M2. (avancé) Réseau de garderies de Deschambault. Le logiciel de gestion écrase la subvention applicable à chaque installation : la fiche dit le taux d'aujourd'hui, et personne ne sait ce qu'elle disait l'an dernier. Le ministère demande un rapport des trois dernières années par taux de subvention. Dites ce que vous pouvez livrer, ce que vous ne pouvez pas, ce que vous mettez en place pour que la question ne se repose pas, et à partir de quand le rapport sera juste.

M3. Transport scolaire de Yamachiche. Le coût par kilomètre existe à trois endroits : dans un modèle dbt, dans une mesure du tableau de bord, et dans un chiffrier que le directeur tient à jour. Les trois donnent des valeurs différentes, et chacun défend la sienne. Décrivez votre démarche, dites où ce calcul doit vivre selon la règle du cours, et ce que vous faites des deux autres.

© 2026 iAlexMG. Tous droits réservés.

Corrigé

Comment lire ce corrigé. Il donne la réponse et le pourquoi, jamais une notion neuve : tout ce qu'il emploie a été enseigné dans la Théorie 05 ou pratiqué dans le Labo 05. Quand une réponse repose sur un geste du labo, le renvoi le nomme.

Questions courtes

1. La définition. Un test dbt est une requête qui ne doit rendre aucune ligne : si elle en rend, le test échoue et les lignes rendues sont les coupables. Ce n'est pas une contrainte de base de données, puisque rien n'empêche l'écriture d'une ligne fautive et que le test la constate après coup ; et ce n'est pas une vérification du code, puisqu'un test de données regarde ce qui est dans la table, pas la logique qui l'y a mise. *Théorie : `Theorie_05_AE.md`, section 1.*

2. Les trois sortes. Le test générique, paramétré et réutilisable, se déclare dans le YAML sous la colonne ou sous le modèle. Le test singulier, écrit pour un cas précis, est un fichier `.sql` dans `tests/`. Le test unitaire se déclare dans le YAML, sous `unit_tests:`. *Théorie : `Theorie_05_AE.md`, section 1.*

3. Le nom du bloc. `data_tests:`, depuis dbt 1.8. L'ancien nom, `tests:`, fonctionne encore et affiche un avertissement de dépréciation à chaque commande, ce qui finit par cacher les vrais messages. La même chose vaut pour les paramètres d'un test paramétré, qui vont sous `arguments:`. *Labo : `Labo_05_AE.md`, étape 1.*

4. Les quatre génériques. `unique`, sur la clé de chaque dimension. `not_null`, sur les clés et sur toute colonne qu'un décodage produit. `accepted_values`, sur le palier et sur la gamme. `relationships`, sur les clés du fait vers ses dimensions. *Théorie : `Theorie_05_AE.md`, section 2.*

5. Le test qui ment. `accepted_values`. Il ignore les valeurs absentes, comme tous les tests génériques de dbt : il vérifie que ce qui est là fait partie de la liste, et une colonne entièrement vide ne contient rien qui n'en fasse pas partie. Mesuré au labo en sabotant le décodage de la gamme : `PASS` sur `accepted_values`, et `FAIL 68` sur `not_null_dim_produit_gamme`, les 68 articles ayant une gamme vide. La parade n'a pas d'exception : un `accepted_values` se pose toujours avec un `not_null` à côté. *Labo : `Labo_05_AE.md`, étape 2.*

6. Le trou rebouché. `relationships` garantit que chaque valeur d'une colonne existe bien dans la colonne d'un autre modèle, donc que chaque clé du fait désigne une ligne de dimension. Le choix qui le rend obligatoire est celui du Cours 04 : le fait recalcule les clés de ses dimensions par hachage au lieu de les joindre, ce qui est rapide et ne garantit rien. Sans ce test, personne ne vérifie jamais que le fait pointe quelque part. *Théorie : `Theorie_05_AE.md`, section 2.*

7. Le grain, une fois pour toutes. `dbt_utils.unique_combination_of_columns`. Il remplace le contrôle du grain que le Cours 04 faisait lancer à la main après chaque construction : la même vérification, écrite une fois dans le YAML et relancée à chaque exécution. *Théorie : `Theorie_05_AE.md`, section 2.*

8. Le test singulier. Il vit dans un fichier `.sql` du dossier `tests/`, et dbt en attend ce qu'il attend de tous les autres : aucune ligne. La réconciliation de Boréal ne peut pas être générique parce qu'elle ne se paramètre pas : elle compare deux populations précises, le fait des ventes et les en-têtes de commande de l'ERP, avec un filtre propre à Boréal sur les commandes entièrement refusées. Un test générique répond à une question générale. *Théorie : `Theorie_05_AE.md`, section 3.*

© 2026 iAlexMG. Tous droits réservés.

9. La jointure aveugle. Une jointure interne ne voit que ce qui existe des deux côtés : un document absent du fait n'entre pas dans la comparaison, ne produit aucune ligne, et le test passe. Mesuré au labo sur un fait amputé de janvier 2022 : la jointure interne rend 0 écart, la jointure externe complète en rend 518. Les trois cas à sortir sont ce qui manque à gauche, ce qui manque à droite, et ce qui existe des deux côtés avec un montant différent. *Labo : `Labo_05_AE.md`, étape 4.*

10. Les trois réglages. `severity`, qui vaut `error` par défaut et peut valoir `warn`, auquel cas le test signale sans arrêter la construction. `warn_if`, la condition qui déclenche l'avertissement. `error_if`, celle qui déclenche l'échec. Les trois se posent sous `config:` du test. *Théorie : `Theorie_05_AE.md`, section 4.*

11. Le seuil. `warn_if: ">23"`, avec `error_if: ">30"`. La valeur vient d'un fait connu : 23 articles n'ont pas d'année-modèle, et ce sont les pièces et accessoires. Ce qu'un seuil documente et qu'un test retiré ne documente pas, c'est ce fait même : la valeur écrite dans le projet est une affirmation datée et versionnée, « il y en a 23, et nous voulons le savoir si ça change ». Un test retiré donne la même tranquillité et aucune information. Vérifié au labo en abaissant le seuil à 22 : `WARN 23 not_null_dim_produit_annee_modele`, avec « Got 23 results, configured to warn if >22 ». *Labo : `Labo_05_AE.md`, étape 5.*

12. Le test unitaire. Ses trois parties : `model`, le modèle testé ; `given`, une liste d'entrées, une par `ref()` ou `source()` que le modèle lit ; `expect`, les lignes attendues en sortie. Il attrape ce qu'aucun test de données n'attrape, la logique d'un modèle aux bornes qui comptent, sans attendre que les données contiennent par hasard le cas limite. Les colonnes que `expect` ne cite pas ne sont pas comparées, ce qui permet de tester une logique sans recopier le modèle entier. *Théorie : `Theorie_05_AE.md`, section 5.*

13. Le modèle invisible. Pour préparer une entrée, dbt va chercher les types des colonnes dans la base ; un modèle `ephemeral` n'y crée aucun objet, donc il n'y a rien où les chercher, et le test s'arrête sur « Not able to get columns for unit test … because the relation doesn't exist ». La parade tient en une ligne : donner cette entrée en `format: sql`, c'est-à-dire par un `select` qui déclare lui-même ses types. *Labo : `Labo_05_AE.md`, étape 9.*

14. Le snapshot. À chaque `dbt snapshot`, dbt compare ce que la source dit maintenant à ce qu'il avait enregistré, ferme les lignes qui ont changé et en ouvre de nouvelles. Ses quatre colonnes : `dbt_scd_id`, l'identifiant de la version ; `dbt_updated_at`, la valeur qui a servi à dater le changement ; `dbt_valid_from`, le début de validité ; `dbt_valid_to`, la fin, vide pour la version courante. *Théorie : `Theorie_05_AE.md`, section 6.*

15. Les deux stratégies. `timestamp` fait confiance à une colonne de date de modification de la source. `check` compare les valeurs d'une liste de colonnes. Boréal emploie `check` sur le palier, parce que le référentiel arrive par extraits mensuels complets : chaque ligne est réécrite à chaque lot, et aucune date de la source ne dit qu'un palier a bougé. C'est le cas courant, et c'est pour lui que la stratégie existe. *Théorie : `Theorie_05_AE.md`, section 6.*

16. La date des versions. Sans `updated_at`, dbt date chaque version de l'heure du lancement : `dbt_valid_from` et `dbt_valid_to` sont des `TIMESTAMP` à la seconde près, et deux ans d'histoire tiennent dans la demi-minute qu'a duré le rejeu. Avec `updated_at='date_extrait'`, la version est datée par la colonne qu'on désigne, ici la fin du mois de l'extrait, et les bornes sont des `DATE` d'affaires. Le cas de Boréal impose la seconde forme, puisqu'on rejoue aujourd'hui une histoire qui a eu lieu entre 2022 et 2025. *Théorie : `Theorie_05_AE.md`, section 6.*

17. La table et la vue. Un snapshot est une table : il ne change pas quand sa source change, il change quand on le lance. Le labo le fait constater en cassant le décodage du palier dans le staging : la dimension et ses tests restent au vert, parce qu'ils lisent le snapshot, qui porte encore les anciennes valeurs. C'est une protection et un piège, la donnée fausse entrant au prochain `dbt snapshot` et pas avant. *Théorie : `Theorie_05_AE.md`, section 6.*

© 2026 iAlexMG. Tous droits réservés.

18. La fréquence. L'extrait du référentiel est mensuel et daté de la fin du mois : le snapshot voit le changement le 31 juillet, parce qu'il n'a regardé que ce jour-là. Les ventes du mois de juillet 2024 chez ces 49 concessionnaires sont donc attribuées à l'ancien palier, 646 lignes pour 4 314 525,62 $. Ce n'est pas une erreur du snapshot, c'est ce qu'il sait : sa précision est celle de sa fréquence. *Théorie : `Theorie_05_AE.md`, section 6.*

19. Les trois décisions du type 2. La clé se hache sur le code et la date d'ouverture de la version, faute de quoi deux versions du même concessionnaire auraient la même clé. Les colonnes qu'on ne suit pas restent en type 1 et viennent de l'état courant du staging, ce qui veut dire qu'une version fermée affiche le nom d'aujourd'hui, et cela se documente. La première version de chaque entité s'ouvre à une date plancher très ancienne, et non à la date où le snapshot l'a vue pour la première fois. *Théorie : `Theorie_05_AE.md`, section 7.*

20. La jointure temporelle. Un intervalle semi-ouvert, fermé au début et ouvert à la fin, parce que dbt ferme une version à la date où la suivante s'ouvre. Le contrôle du grain attrape le dédoublement d'un intervalle fermé des deux côtés, puisqu'une vente du jour de bascule correspond alors aux deux versions. La réconciliation attrape les lignes perdues, qu'un contrôle de grain ne peut pas voir. Aucun des deux n'attrape le défaut de l'autre, et c'est la raison pour laquelle on en pose deux. *Théorie : `Theorie_05_AE.md`, section 8.*

21. Ce qui change et ce qui ne change pas. L'historisation change l'attribution de la valeur, jamais son total. Chez Boréal, 17 733 lignes de vente changent de palier selon qu'on lit la version d'époque ou celle d'aujourd'hui, pour 94 628 792,88 $, et le total reste à 430 148 012,16 $, à 0,00 $ près de la valeur portée par l'ERP. *Théorie : `Theorie_05_AE.md`, section 8.*

22. La documentation. `manifest.json`, tout ce que dbt sait du projet, graphe et SQL compilé compris ; `catalog.json`, ce que la base sait des objets produits, colonnes et types ; `index.html`, le site qui lit les deux. Le `manifest.json` ressert deux fois dans le parcours : au Cours 08, pour qu'un agent d'intelligence artificielle sache ce que l'entrepôt contient, et dans la formation Architecture et Gouvernance, pour remonter d'un rapport jusqu'à la source. C'est le seul endroit où le lignage existe sous une forme qu'un programme peut lire. *Théorie : `Theorie_05_AE.md`, section 9.*

23. L'`exposure` et le `rpt_`. Un `exposure` déclare ce qui vit en aval du projet, un tableau de bord par exemple, avec son type, ses dépendances et son propriétaire, et il entre dans le graphe comme un nœud terminal. Son propriétaire est une équipe et une adresse de groupe, jamais une personne nommée, qui périme au premier départ. Un modèle `rpt_*` a le droit de faire trois choses : joindre le fait à ses dimensions, agréger à un grain plus grossier, et rien d'autre. Il ne crée aucune mesure que l'étoile ne porte pas et ne corrige rien. *Théorie : `Theorie_05_AE.md`, sections 10 et 11.*

Questions pratiques

P1. Tester une dimension qui a changé de forme. Un `unique` posé sur `code_concessionnaire` échouerait, et c'est normal : la dimension porte une ligne par version, donc le code n'y est plus unique. dbt rend une ligne par valeur fautive.

```sql
select code_concessionnaire, count(*) as versions
from marts.dim_concessionnaire
group by 1
having count(*) > 1;
```

Attendu : 49 valeurs, les 49 concessionnaires dont le palier a changé, pour 98 des 205 versions.

© 2026 iAlexMG. Tous droits réservés.

Le grain réel de la dimension est le couple du code et de la date d'ouverture, et c'est lui qu'on teste. Dans `_marts.yml`, sous le modèle et non sous une colonne :

```yaml
    data_tests:
      - dbt_utils.unique_combination_of_columns:
          arguments:
            combination_of_columns:
              - code_concessionnaire
              - valide_de
```

L'`unique` ne disparaît pas pour autant : il se déplace sur `cle_concessionnaire`, la clé de la version, qui doit rester unique. Les deux tests ne disent pas la même chose, et il faut les deux : l'un vérifie que la clé technique est bien une clé, l'autre que le grain déclaré est le vrai. *Labo : `Labo_05_AE.md`, étape 7.*

P2. La réconciliation.

```sql
with fait as (

    select numero_commande, sum(valeur_nette) as valeur
    from {{ ref('fct_ventes') }}
    group by 1

),

erp as (

    select numero_commande, valeur_nette as valeur
    from {{ ref('stg_commandes') }}
    where valeur_nette <> 0

)

select
    coalesce(fait.numero_commande, erp.numero_commande) as numero_commande,
    fait.valeur as valeur_du_fait,
    erp.valeur as valeur_de_l_erp

from fait
full outer join erp on erp.numero_commande = fait.numero_commande

where fait.numero_commande is null
   or erp.numero_commande is null
   or abs(fait.valeur - erp.valeur) > 0.005
```


© 2026 iAlexMG. Tous droits réservés.

Ses trois particularités règlent trois choses. Le `full outer join` fait voir ce qui manque d'un côté comme de l'autre, qu'une jointure interne laisse passer en silence. Le filtre `valeur_nette <> 0` écarte les 227 commandes entièrement refusées, dont l'ERP porte une valeur nulle et que le fait ne contient pas, sans quoi le test échouerait tous les jours sur un écart qui n'en est pas un. La tolérance de `0.005` compare des décimaux sans se faire piéger par le dernier cent.

Attendu : aucune ligne rendue, donc le test au vert. Et la preuve de l'aveuglement, sur un fait amputé de janvier 2022 : 0 écart vu par la jointure interne, 518 par la jointure externe complète. *Labo : `Labo_05_AE.md`, étape 4.*

P3. Le coût du type 1, mesuré après coup.

```sql
with courant as (

    select code_concessionnaire, palier
    from marts.dim_concessionnaire
    where est_courante

)

select
    d.annee,
    count(*) as lignes,
    sum(f.valeur_nette) as valeur_nette

from marts.fct_ventes as f
inner join marts.dim_concessionnaire as v on v.cle_concessionnaire = f.cle_concessionnaire
inner join courant as c on c.code_concessionnaire = v.code_concessionnaire
inner join marts.dim_date as d on d.date_jour = f.cle_date
where v.palier <> c.palier
group by 1
order by 1;
```

Année	Lignes	Valeur nette
2022	6 224	30 991 592,74 $
2023	7 107	38 323 089,14 $
2024	4 402	25 314 111,00 $

Soit 17 733 lignes et 94 628 792,88 $ en tout. Ce que la dernière année apprend est ce qui manque au tableau : 2025 n'y est pas, avec zéro ligne. La révision des paliers date du 1er juillet 2024, donc plus rien ne diffère ensuite, et une dimension de type 1 répond juste à toute question qui porte sur la période récente. C'est ce qui rend ce genre de défaut si durable : il ne se voit pas dans les chiffres du mois, seulement dans ceux d'avant. *Théorie : `Theorie_05_AE.md`, section 7.*

P4. Un test de plus. Dans `tests/assert_versions_continues.sql` :

© 2026 iAlexMG. Tous droits réservés.

```sql
with bornes as (

    select
        code_concessionnaire,
        valide_de,
        valide_jusqua,
        lead(valide_de) over (
            partition by code_concessionnaire
            order by valide_de
        ) as debut_de_la_suivante

    from {{ ref('dim_concessionnaire') }}

)

select
    code_concessionnaire,
    valide_de,
    valide_jusqua,
    debut_de_la_suivante

from bornes
where debut_de_la_suivante is not null
  and debut_de_la_suivante <> valide_jusqua
```

Attendu : aucune ligne. Une version doit finir exactement là où la suivante commence, et la seule comparaison à faire est celle-là : plus grand, c'est un trou ; plus petit, c'est un chevauchement.

Ce que les tests déjà en place feraient de ces deux défauts est instructif, et c'est la raison d'écrire celui-ci. Aucun des deux ne serait vu dans la dimension : les clés restent uniques, le grain reste bon, les paliers restent dans la liste. Ils ne se verraient que plus loin, chacun par un détour. Un chevauchement dédoublerait des lignes du fait, et le contrôle du grain de `fct_ventes` finirait par tomber ; un trou en ferait disparaître, et la réconciliation tomberait. Un test qui attrape un défaut à l'endroit où il naît vaut mieux qu'un test qui l'attrape deux modèles plus loin, sous un autre nom.

P5. (avancé) Le test unitaire du plancher. Il faut trois entrées, une par `ref()` que `dim_concessionnaire` lit : le snapshot, `stg_concessionnaires` et `stg_regions`. La logique testée n'en concerne qu'une, mais dbt construit le modèle en entier pour le comparer, et un modèle ne se construit pas avec des entrées manquantes.

```yaml
  - name: test_plancher_de_la_premiere_version
    description: "La première version s'ouvre au plancher, les suivantes à leur date."
    model: dim_concessionnaire
    given:
      - input: ref('snp_concessionnaire')
```


© 2026 iAlexMG. Tous droits réservés.

```
      rows:
        - {code_concessionnaire: 'D1', palier: 'Bronze',
           dbt_valid_from: '2022-01-31', dbt_valid_to: '2024-07-31'}
        - {code_concessionnaire: 'D1', palier: 'Argent',
           dbt_valid_from: '2024-07-31', dbt_valid_to: null}
    - input: ref('stg_concessionnaires')
      rows:
        - {code_concessionnaire: 'D1', nom_concessionnaire: 'Essai', ville: 'Québec',
           province: 'QC', date_creation: '2022-01-05', est_supprime: false}
    - input: ref('stg_regions')
      rows:
        - {code_province: 'QC', nom_province: 'Québec'}
  expect:
    rows:
      - {palier: 'Bronze', valide_de: '1900-01-01', est_courante: false}
      - {palier: 'Argent', valide_de: '2024-07-31', est_courante: true}
```

Le test passe en moins d'une seconde. Quand une entrée manque, dbt ne devine pas : il s'arrête sur « Unit_Test '…' depends on a node named 'stg_regions' which was not found ». Le message est clair, ce qui n'est pas toujours le cas, et il nomme l'entrée oubliée.

Ce test vaut surtout par le moment où il tombe. Retirez le plancher du modèle et lancez `dbt build` : il échoue sur la dimension, avant que le fait soit construit, et tout l'aval est sauté. La réconciliation aurait fini par signaler le même défaut, mais après coup, sur un fait déjà écrit et sur un écart de plusieurs millions qu'il aurait fallu expliquer. *Théorie : `Theorie_05_AE.md`, section 5.*

P6. Ce que je casse, et d'où ça vient.

```
dbt ls --select dim_concessionnaire+
dbt ls --select +tableau_de_bord_des_ventes
```

Les comptes qui suivent sont ceux du projet à la fin du labo, avant les tests ajoutés par les questions P4 et P5.

La première rend 20 nœuds : la dimension, le fait, le résumé, l'`exposure`, et les quinze tests et le test unitaire qui portent dessus. C'est la réponse à « qu'est-ce que je casse si je touche à ce modèle », et c'est aussi la sélection à lancer après l'avoir touché.

La seconde rend 42 nœuds : l'`exposure` lui-même, onze modèles, le snapshot, cinq sources, les 23 tests de données et le test unitaire. C'est la réponse à « de quoi ce tableau de bord dépend-il », et la sélection à lancer avant une publication, sous la forme `dbt build --select +tableau_de_bord_des_ventes`.

Le douzième modèle du projet, `ctl_commandes_sans_concessionnaire`, n'apparaît pas dans la seconde liste : il ne mène à rien, et c'est normal, un modèle de contrôle n'est pas censé nourrir un rapport. *Théorie : `Theorie_05_AE.md`, section 10.*

P7. Le taux de marge par palier.

© 2026 iAlexMG. Tous droits réservés.

```sql
select
    palier,
    sum(lignes) as lignes,
    sum(valeur_nette) as valeur_nette,
    sum(marge) as marge,
    round(100.0 * sum(marge) / sum(valeur_nette), 2) as taux_de_marge
from presentation.rpt_ventes_par_palier
where annee = 2025
group by 1
order by 1;
```

Palier	Lignes	Valeur nette	Marge	Taux de marge
Argent	7 366	44 285 006,31 $	12 128 582,63 $	27,39 %
Bronze	5 603	30 468 009,66 $	8 920 130,14 $	29,28 %
Or	6 040	42 516 351,43 $	10 848 466,03 $	25,52 %

La moyenne des douze taux mensuels donne 27,69, 29,69 et 25,96 pour cent. Le juste est le premier tableau, celui qui divise la somme des marges par la somme des valeurs nettes. L'écart est ici de trois dixièmes de point seulement, parce que les douze mois d'un même palier se ressemblent ; il était de onze points au Cours 04, sur des lignes de tailles très différentes. C'est ce qui rend la faute difficile à voir : elle donne souvent un chiffre plausible, et elle explose quand on ne s'y attend pas. La règle ne dépend donc pas de l'ampleur de l'écart, elle est de méthode : on garde le numérateur et le dénominateur, le ratio se calcule au niveau où il s'affiche. *Théorie : `Theorie_05_AE.md`, section 11.*

P8. (avancé) Rejouer les quarante-neuf lots. Les lots sont les valeurs distinctes de `_lot` dans `raw.erp_kna1`, de `2022-01` à `2025-12`, plus `2023-03_bis`. On les rejoue dans l'ordre, par un `dbt snapshot --vars "{lot: …}"` par lot, boucle de quarante-neuf commandes.

Ce que cela change dans le snapshot : les dates d'ouverture de cinq concessionnaires, ceux qui sont apparus après le premier lot. Avec quatre lots, ils s'ouvrent au lot où le snapshot les a vus pour la première fois, `2024-06` ou `2025-12` ; avec les 49, à leur vrai mois, du 2022-12-31 au 2025-09-30. Les comptes, eux, ne bougent pas : 205 lignes, 156 concessionnaires, 49 versions fermées, toutes au 31 juillet 2024.

Ce que cela change dans l'étoile finale : rien. Le fait rend les mêmes 75 652 lignes pour 430 148 012,16 $, et le partage de 2023 par palier tombe au cent près sur les mêmes valeurs. La raison est le plancher : la première version de chaque concessionnaire s'ouvre en 1900 dans les deux cas, donc la date où le snapshot l'a découvert n'a aucun effet sur l'attribution des ventes. Une décision de modélisation rend le résultat insensible à la finesse du rejeu, et c'est ce qui justifie de ne pas payer les 49 lots.

Ce que cela coûte : 528 secondes mesurées, contre quatre commandes en une quarantaine de secondes. Chaque `dbt snapshot` paie le démarrage de dbt, une dizaine de secondes, pour un travail qui prend moins d'une seconde. C'est le genre de calcul qu'on refait quand l'histoire à rejouer compte non plus 49 lots mais 3 000.

© 2026 iAlexMG. Tous droits réservés.

Diagnostics

E1. L'histoire datée d'aujourd'hui. Le snapshot porte bien 205 lignes et 49 versions fermées, mais toutes ses bornes sont des `TIMESTAMP` de la journée du rejeu, à la seconde près : les quatre lots se sont exécutés en une trentaine de secondes, et deux ans d'histoire y tiennent. Mesuré : une première version ouverte à 23:36:26, la révision de juillet 2024 datée de 23:36:47.

La conséquence dans le fait est complète mais silencieuse. Toute vente est antérieure au jour du rejeu, donc toute vente tombe dans la première version de son concessionnaire, celle qui porte l'ancien palier. Le tableau des ventes 2025 par palier annonce 45 204 154,23 $ pour l'Or, 39 249 387,29 $ pour l'Argent et 32 815 825,88 $ pour le Bronze, là où les vraies valeurs sont 42 516 351,43 $, 44 285 006,31 $ et 30 468 009,66 $. Cinq millions de dollars changent de palier sur le seul exercice 2025.

Aucun test ne l'a vu, et c'est logique : le total est juste, le grain est juste, chaque clé du fait désigne bien une ligne de la dimension, et la réconciliation compare des montants, pas des attributions. L'erreur porte sur l'appariement d'une vente à une version, et il n'existe aucun témoin indépendant de cet appariement. Ce qui l'aurait vue, c'est un regard sur le snapshot lui-même : une histoire d'entreprise dont toutes les dates tombent le même après-midi n'est pas une histoire d'entreprise. *Théorie : `Theorie_05_AE.md`, section 6.*

E2. Le lot rejoué trop tard. Le `dbt build` a créé le snapshot avec l'état courant, celui du lot le plus récent, et lui a donné une seule version par concessionnaire, datée du 2025-12-31. Le lot `2024-06` arrive ensuite : la stratégie `check` compare le palier du lot à celui qu'elle a en mémoire, trouve une différence pour les 49 concessionnaires révisés, ferme leur version courante et en ouvre une neuve datée du 2024-06-30.

Le résultat est le pire possible, parce qu'il a l'air juste. Mesuré : 205 lignes, 156 concessionnaires, 156 versions ouvertes, 49 fermées, exactement les comptes que le labo annonce. Ce qui cloche ne se voit que dans les bornes : 49 versions vont du 2025-12-31 au 2024-06-30, une fin antérieure à son début, et la version « courante » de ces 49 concessionnaires porte le palier de juin 2024. Le fait, lui, attribue tout à cette version courante, et le tableau 2025 est faux des mêmes millions qu'au diagnostic précédent.

Ce qui produit ce résultat est la mécanique même de la stratégie `check` : elle ne compare pas des dates, elle compare des valeurs à ce qu'elle a enregistré la dernière fois, et elle date la nouvelle version avec ce que la requête lui donne. Elle n'a aucun moyen de savoir qu'on lui présente un passé. C'est pourquoi un snapshot se rejoue dans l'ordre, et pourquoi le labo exclut `snp_concessionnaire` de `dbt build` tant que l'histoire n'est pas montée. *Labo : `Labo_05_AE.md`, étape 6.*

E3. Les lignes qui manquent, puis celles qui restent. Premier temps, l'intervalle semi-ouvert sans plancher : les ventes antérieures à la première version d'un concessionnaire ne trouvent personne à qui se joindre, et la jointure interne les fait disparaître. Mesuré : `PASS=32 WARN=0 ERROR=1 SKIP=3`, un seul test tombe, `assert_reconciliation_ventes`, sur 633 commandes en écart. Les 1 894 lignes disparues ne violent aucune contrainte : les clés restantes sont uniques, pointent toutes une dimension, et le grain est parfait. Un fait amputé est un fait bien formé.

Second temps, le `left join` : les lignes reviennent, avec une clé de concessionnaire vide. La réconciliation repasse au vert, puisque les montants sont de nouveau tous là, et c'est `not_null_fct_ventes_cle_concessionnaire` qui tombe, sur 1 894 lignes. Le `relationships` aurait laissé passer, lui, puisqu'il ignore les valeurs absentes comme les autres tests génériques.

L'histoire enseigne que les tests ne se remplacent pas les uns les autres. Le même défaut, la première version qui s'ouvre trop tard, se manifeste de deux façons selon la jointure choisie, et chaque fois un seul test le voit, jamais le même. Deux tests portent pourtant sur cette colonne, et à chaque tour l'un des deux est

© 2026 iAlexMG. Tous droits réservés.

resté muet. Ce n'est pas une raison d'en retirer un, c'est la raison de les garder tous les deux. La vraie correction n'est d'ailleurs ni la jointure ni le test : c'est le plancher de l'étape 7. *Labo : `Labo_05_AE.md`, étape 8.*

E4. Le rouge qu'on met en jaune. dbt affiche `WARN 633 assert_reconciliation_ventes`, puis « Completed with 1 warning » et « Got 633 results, configured to warn if != 0 ». Le compte final se lit `PASS=35 WARN=1 ERROR=0`, et le code de sortie du processus est zéro.

Ce que l'ordonnanceur en conclut est donc que tout va bien, et il a raison de le conclure : c'est exactement ce qu'on lui a dit. Trois semaines de nuits vertes, avec un écart de plusieurs millions sur le chiffre d'affaires, et un avertissement noyé dans un journal que personne ne lit une fois que la tâche est au vert. Un avertissement qui ne réveille personne n'est pas un avertissement, c'est un test désactivé avec des étapes en plus.

Ce qu'il fallait faire à la place dépend de ce qu'on sait. Si l'écart est connu, borné et accepté, il prend un seuil chiffré, `error_if` compris, et le chiffre documente le fait, comme pour `annee_modele`. S'il n'est pas compris, le test reste en erreur, la construction reste rouge, et c'est le bon état : une étoile dont on ne sait pas expliquer six millions ne doit pas se livrer en silence. Entre les deux, il reste la voie honnête, qui est de retirer du fait ce qu'on ne sait pas expliquer plutôt que de baisser le seuil de ce qui le mesure. *Théorie : `Theorie_05_AE.md`, section 4.*

E5. La dimension qui triple. Le snapshot est passé de 205 à 614 lignes. `date_extrait` change à chaque lot par construction, puisqu'elle porte la fin du mois de l'extrait : la stratégie `check` y voit un changement à chaque passage et ouvre une version par concessionnaire et par lot. Quatre lots, 614 lignes ; les 49 vraies versions sont noyées parmi 409 qui ne disent rien.

Rien ne casse, et c'est ce qui rend le cas intéressant. Les intervalles restent disjoints et contigus, donc le grain du fait tient ; chaque vente trouve une version et une seule, donc les clés sont bonnes ; et comme le palier est le même de part et d'autre d'une bascule de date d'extrait, le chiffre d'affaires par palier est identique au cent près. Tous les tests passent.

Le défaut se voit à deux endroits, et aucun n'est un test. Dans la taille de la table d'abord, qui triple sans qu'aucune information n'apparaisse, et qui triplerait encore à chaque lot mensuel. Dans la lecture ensuite : la question à laquelle une dimension de type 2 doit répondre est « qu'est-ce qui a changé, et quand », et une table où chaque ligne change tous les mois ne répond plus à cette question. La règle qui tranche est simple : on suit les colonnes dont un changement a un sens pour le métier, jamais celles qui bougent parce que le fichier est neuf. *Théorie : `Theorie_05_AE.md`, section 6.*

Questions à développer

A. Combien de tests est-ce assez. Le nombre se défend d'abord par ce qu'il coûte, qui est presque rien : le projet entier se construit et se vérifie en moins de deux secondes, dans une seule commande. Un test qui ne coûte rien et qui peut éviter une livraison fausse n'a pas à se justifier par un calcul de rentabilité.

Il se défend surtout par ce que chacun couvre. Ils ne sont pas vingt-quatre variantes de la même vérification : il y a les clés, qui vérifient que la dimension est une dimension ; le grain, qui vérifie que le fait dit ce que son modèle annonce ; les relations, qui rebouchent le trou laissé par le choix de recalculer les clés ; les valeurs acceptées, qui attrapent un décodage qui a changé ; la réconciliation, qui est le seul témoin indépendant du projet ; et le test unitaire, seul à regarder une logique plutôt qu'un résultat.

© 2026 iAlexMG. Tous droits réservés.

Ce qui distingue un test qui gagne sa place, c'est qu'il puisse échouer un jour pour une raison qu'on ne sait pas encore. Un test dont on est certain qu'il ne tombera jamais ne vaut pas sa ligne : il occupe le tableau de bord, allonge l'exécution, et surtout il habitue à voir du vert. Le contraire est le vrai risque des suites de tests qui grossissent : vingt-quatre tests qu'on lit, ou deux cents qu'on ne lit plus et dont trois sont rouges depuis six mois.

Ce que ces tests ne vérifient toujours pas mérite d'être écrit, parce que c'est ce qu'un lecteur du `LIVRAISON.md` a le droit de savoir. Ils ne vérifient rien sur la fraîcheur : l'étoile passerait au vert sur une `RAW` vieille de six mois. Ils ne vérifient pas que les lignes du fait sont les bonnes lignes, seulement que leur total concorde avec l'ERP. Ils ne disent rien des attributions, comme les deux premiers diagnostics l'ont montré. Et aucun ne compare deux exécutions successives, donc une chute de moitié du volume passerait sans bruit.

B. Type 1 ou type 2. Historiser toutes les colonnes coûte trois choses, et aucune n'est visible le jour où on décide. La table grossit d'une ligne par changement, et une colonne qui bouge souvent, une adresse de courriel, un numéro de téléphone, suffit à multiplier les versions sans rien apprendre à personne, comme le diagnostic E5 l'a montré avec une seule colonne volatile. Le modèle se complique, puisque chaque jointure vers la dimension doit désormais porter une condition de date. Et surtout, la lecture se brouille : une dimension où tout est historisé ne dit plus ce qui compte, et la question « qu'est-ce qui a changé » n'a plus de réponse courte.

Ce qui décide qu'une colonne mérite d'être suivie tient en une question de métier : est-ce qu'une analyse a un sens différent selon la valeur d'époque et la valeur d'aujourd'hui. Le palier du concessionnaire la passe, puisque « combien avons-nous vendu aux Or en 2023 » veut dire quelque chose et n'a pas la même réponse selon la lecture. Le numéro de téléphone ne la passe pas : personne n'analyse des ventes par numéro de téléphone d'époque. Entre les deux, le nom et la ville sont des cas d'arbitrage, et l'arbitrage se fait avec ceux qui poseront les questions, pas seul.

Ce qu'on écrit pour que le choix reste défendable dans deux ans tient en deux lignes dans la description de la dimension : quelles colonnes sont historisées, et que les autres portent la valeur d'aujourd'hui, y compris sur les versions fermées. La deuxième phrase est celle qui compte, parce qu'elle est contre-intuitive : une ligne qui dit « valide jusqu'au 31 juillet 2024 » et qui affiche le nom de 2026 surprendra quelqu'un un jour, et il vaut mieux qu'il lise la réponse plutôt qu'il ouvre un billet.

C. (avancé) Le jour où les clés changent. Ce qui casse est net et total : les 205 clés de la dimension sont neuves, et aucune ne vaut plus le `md5` du code. Mesuré : zéro version dont la clé soit restée la même, et zéro ligne du fait, sur 75 652, qui joindrait encore avec une clé calculée à l'ancienne. Tout ce qui, en aval, avait recalculé la clé plutôt que de la lire, un modèle, un rapport, un chiffrier, se retrouve avec une jointure vide. Ce qui casse aussi, plus discrètement, c'est le sens de la clé : elle désignait un concessionnaire, elle désigne maintenant une version, et un rapport qui comptait des clés distinctes compte désormais 205 concessionnaires au lieu de 156.

Ce qui ne casse pas est tout le reste : les mesures, les dates, les dimensions dégénérées, les totaux. Le fait porte les mêmes 75 652 lignes pour les mêmes 430 148 012,16 $. Le code du concessionnaire est toujours là, en clair, dans la dimension, et c'est par lui que tout se rebranche.

Mener ce changement demande de traiter la clé comme une interface, ce qu'elle est. Concrètement : prévenir avant, en nommant ce qui change et ce qui ne change pas, plutôt que d'annoncer une amélioration ; chercher qui consomme, et l'`exposure` est fait pour ça, à condition d'avoir été tenu à jour ; livrer d'abord la dimension avec sa nouvelle clé sans retirer l'ancienne colonne, pour que les deux mondes coexistent le temps d'un cycle ; et publier la façon de joindre, l'intervalle semi-ouvert compris, parce que ceux qui vont écrire cette jointure ne sont pas ceux qui ont lu ce cours. Ce qui ne marche pas, c'est de livrer un mardi soir en comptant sur les tests : ils étaient tous au vert, et c'est bien le problème, aucun test du projet ne sait ce que quelqu'un d'autre fait de ses clés.

© 2026 iAlexMG. Tous droits réservés.

Mises en situation

M1. Distribution alimentaire de Saint-Raymond. Je pose d'abord les tests qui disent si la table est ce qu'elle prétend être, parce qu'ils ne demandent aucune connaissance du métier et qu'ils attrapent les défauts les plus graves : `unique` et `not_null` sur la clé de chaque dimension, le contrôle du grain sur le fait, puis `relationships` de chaque clé du fait vers sa dimension. C'est une heure de travail, et cela répond à « l'étoile tient-elle debout ».

Je pose ensuite le seul test qui demande de comprendre l'entreprise, et c'est celui qui vaut tous les autres : la réconciliation contre un témoin indépendant. Je cherche un total que le système source calcule lui-même, ou à défaut celui que la comptabilité publie déjà, et je le compare en jointure externe complète. S'il n'existe aucun témoin, je le dis, et cela devient la première ligne de ce que je rends.

Ce que je ne pose pas tout de suite, ce sont les `accepted_values` et les seuils. Ils exigent de connaître les valeurs légitimes et les exceptions normales, et une liste devinée en une journée produira des échecs quotidiens que l'équipe apprendra à ignorer, ce qui est pire que pas de test du tout. Je note les colonnes candidates et je les traite avec quelqu'un qui connaît les données.

Ce que je rends à la fin de la journée est en deux parties. Le projet avec ses tests, qui tournent dans la même commande que la construction, et dont je donne le compte et le temps. Et une page courte qui dit ce qui est vérifié, ce qui ne l'est pas encore, et ce que j'ai trouvé en chemin, avec les chiffres. Si un test a échoué dès le premier passage, il ne se corrige pas en cachette : il est le résultat le plus utile de la journée.

M2. (avancé) Réseau de garderies de Deschambault. Ce que je peux livrer tout de suite, c'est le rapport des trois ans avec le taux d'aujourd'hui appliqué à toute la période, à condition de le dire en toutes lettres et de chiffrer ce que cela représente : combien d'installations ont un taux qui a changé, et quelle part des montants passés s'en trouve mal attribuée. Chez Boréal, la même question valait 94 628 792,88 $ sur 17 733 lignes, et c'est ce genre de chiffre qui permet au ministère de décider si le rapport lui suffit.

Ce que je ne peux pas livrer, c'est l'histoire elle-même. Un snapshot ne reconstruit pas le passé, il commence à regarder le jour où on le lance. Si le système source écrase, et qu'aucune archive n'existe, alors l'information n'existe nulle part et aucune technique ne la fera apparaître.

Avant de conclure à l'impossible, je cherche pourtant trois choses, dans cet ordre : une table d'historique dans le système source, que les progiciels tiennent souvent sans le dire ; des extraits anciens quelque part, un envoi mensuel au ministère, une sauvegarde, un chiffrier de suivi ; et les documents officiels, puisqu'une subvention se décide par une lettre datée. La troisième voie donne souvent le meilleur résultat, et c'est un chargement manuel dans une table, pas un snapshot.

Ce que je mets en place pour que la question ne se repose pas est un snapshot sur la source, en stratégie `check` sur la colonne du taux, lancé à la même fréquence que les changements possibles. Je le lance le jour même, avant toute autre chose, parce que chaque semaine d'attente est une semaine d'histoire perdue. Et j'écris que le rapport sera juste à partir de cette date, pour la période qui suit seulement : une phrase datée, dans la livraison, qui dit exactement à partir de quand la réponse change de nature.

M3. Transport scolaire de Yamachiche. Ma démarche commence par ce qui n'est pas technique : je demande à chacun la définition qu'il emploie. Trois valeurs différentes viennent presque toujours de trois définitions différentes, et non de trois erreurs de calcul. Les kilomètres à vide comptent-ils, le salaire du conducteur est-il dedans, l'amortissement des véhicules aussi, et sur quelle période. Tant que ce n'est pas écrit, comparer les chiffres ne sert à rien.

© 2026 iAlexMG. Tous droits réservés.

Une fois les définitions posées, je regarde d'où vient chaque calcul et je les recompte sur une même période courte. Il arrive que deux définitions supposément identiques divergent quand même, et c'est alors une vraie erreur, qui se trouve en comparant les populations, comme une réconciliation : ce que l'un compte et pas l'autre.

Où le calcul doit vivre, la règle du cours le dit : le coût par kilomètre est un ratio, donc il ne se stocke pas, il se calcule au niveau où il s'affiche. Ce qui vit dans dbt, ce sont le numérateur et le dénominateur, le coût total et les kilomètres, au grain le plus fin disponible, avec les règles de métier qui décident ce qui entre dans chacun. Le modèle `rpt_*` les agrège au grain du rapport, le tableau de bord fait la division, et personne ne stocke jamais le ratio.

Des deux autres, la mesure du tableau de bord se garde et s'aligne sur cette définition, puisque c'est là que le calcul doit se faire. Le chiffrier du directeur ne se supprime pas par décret : il existe parce qu'il répond à un besoin, souvent une ventilation que le tableau de bord n'offre pas. Je regarde ce qu'il fait de plus, je le fais entrer dans le modèle, et je propose de le remplacer seulement quand il n'apporte plus rien. Et je déclare le tableau de bord en `exposure`, pour que la prochaine fois qu'une colonne bouge, la question « qui consomme ceci » ait une réponse dans le projet plutôt que dans les souvenirs de l'équipe.

© 2026 iAlexMG. Tous droits réservés.